



Data Analytics Project: Cars Data Analysis

Dataset Used: cars_data.csv

Team 3

- 1 Narayana Rao Mungam
- 2 Sravana Sandhya Nalabolu
- 3 Dinesh kumar Vangari
- 4 Chaitanya Mandhrapu

This document outlines a structured, end-to-end data analysis project based on railway gauge data. The project is divi

Data Analytics Project: Cars Data Analysis

This document outlines a structured, end-to-end data analysis project based on railway gauge data.

Dataset: Cars Data (Assumed to be in a CSV file)

Project Goal: To analyze seller distributions on various cars and selling price trends.

Modules Used

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling, Type Conversion
Pandas Operations	Filtering, Selection, Aggregation, Feature Engineering
NumPy Calculations	Yearly growth calculation (np.diff)
Visualization	Line Graph, Bar Chart, Pie Chart, Stacked Bar Chart

Scenario 1: Data Loading & Basic Cleaning (□ Easy)

Description: This project analyzes car data using Python libraries like Pandas, NumPy, and Matplotlib. The dataset is loaded and explored to understand its structure and features. Data cleaning is performed by converting columns into numeric format and handling missing values. Basic statistical calculations such as minimum, maximum, and average selling price are computed. A line graph is generated to visualize the selling price trend of the first 10 cars. This helps in understanding price variations clearly.

Task	Description
1.	Load the dataset into a Pandas DataFrame.
2.	Display the first 5 rows, last 5 rows, column names and Shape of dataset
3.	Check the data types of all columns.
4.	Check for missing values in Selling_Price, Present_Price, Kms_Driven, Fuel_Type
5.	Fill missing values: Selling_Price → mean, Present_Price → mean, Kms_Driven → mean, Fuel_Type → mode
6.	Convert numeric columns to proper numeric type if required: Selling_Price, Present_Price, Kms_Driven, Year
7.	Convert Selling_Price and Kms_Driven into NumPy arrays.
8.	Use NumPy to calculate: minimum selling price, maximum selling price, average selling price

Results of Scenario 1:

```
===== RESTART: C:/Analytics/Project 5/test.py =====
FIRST 5 ROWS:
  Car_Name  Year  Selling_Price  ...  Seller_Type  Transmission  Owner
0    ritz    2014         3.35  ...      Dealer      Manual      0
1    sx4    2013         4.75  ...      Dealer      Manual      0
2    ciaz    2017         7.25  ...      Dealer      Manual      0
3  wagon r    2011         2.85  ...      Dealer      Manual      0
4    swift    2014         4.60  ...      Dealer      Manual      0

[5 rows x 9 columns]

LAST 5 ROWS:
  Car_Name  Year  Selling_Price  ...  Seller_Type  Transmission  Owner
296    city    2016         9.50  ...      Dealer      Manual      0
297    brio    2015         4.00  ...      Dealer      Manual      0
298    city    2009         3.35  ...      Dealer      Manual      0
299    city    2017        11.50  ...      Dealer      Manual      0
300    brio    2016         5.30  ...      Dealer      Manual      0

[5 rows x 9 columns]

COLUMNS:
Index(['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven',
       'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner'],
      dtype='str')

SHAPE:
(301, 9)

DATA TYPES:
Car_Name      str
Year          int64
Selling_Price float64
Present_Price float64
Kms_Driven    int64
Fuel_Type     str
Seller_Type   str
Transmission  str
Owner        int64
dtype: object
```

Conclusion: The project successfully demonstrates how to process and analyze car data using Python. Data cleaning improves the quality and accuracy of the dataset. Statistical analysis provides useful insights into selling prices. The visualization makes it easier to interpret trends and patterns. Overall, the project shows how data analysis techniques can support better decision-making in real-world scenarios.

Scenario 2: Selling Price Trend (Line Graph) (□ Easy)

Description: In this scenario, the dataset is used to analyze the selling price trend of cars. The columns Car_Name and Selling_Price are selected, and only the first 10 records are considered for simplicity. The selling prices are converted into a NumPy array for efficient processing. A line graph is plotted using Matplotlib where the x-axis represents the index (0–9) and the y-axis represents the selling price. Markers are added to clearly visualize each data point. This helps in understanding how prices vary across the selected cars.

Task	Description
1.	Select Car_Name and Selling_Price
2.	Take the first 10 rows only using Pandas. Convert Selling_Price into a NumPy array.
3.	Plot a line graph using Matplotlib: X-axis → row index (0–9) , Y-axis → Selling Price Add: ◦ title ◦ x-axis label ◦ y-axis label ◦ markers
4.	Save the graph with a suitable filename

Graph :Selling Price Trend (First 10 Cars)



Conclusion: The line graph shows noticeable variation in selling prices among the first 10 cars. Some cars have significantly higher prices while others are comparatively lower, indicating diversity in car value. There is no consistent increasing or decreasing trend, which suggests that selling price depends on multiple factors like model, year, and condition. The visualization makes it easy to identify price fluctuations. Overall, this scenario helps in quickly understanding price distribution in a small sample.

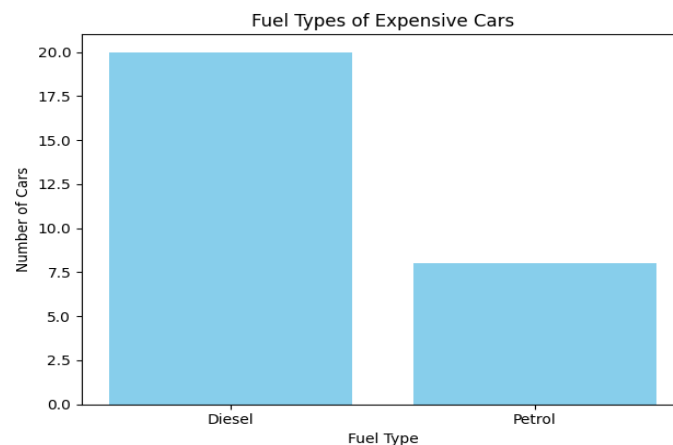
Scenario 3: Expensive Cars Analysis Filtering + Bar Chart (□ Medium)

Description:

This analysis focuses on the luxury segment of the automotive market by filtering the dataset for vehicles with a Selling Price greater than 10 units. By grouping this high-end subset by Fuel Type and counting the occurrences, we identify which engine technologies are preferred for expensive models. The resulting data is converted into NumPy arrays for efficient handling and visualized through a Matplotlib bar chart. This chart clearly displays the distribution across categories like Petrol, Diesel, and CNG, providing a visual tally of which fuel source powers the majority of high-value cars in the inventory

Task	Description
1.	Filter cars where: Selling_Price > 10
2.	Group the filtered data by: Fuel_Type
3.	Count number of cars in each fuel type.
4.	Convert: fuel type labels, counts into NumPy arrays. Plot a bar chart using Matplotlib: X-axis → Fuel Type, Y-axis → Count of expensive cars
5.	Add: title, x-label, y-label Save the graph.

Graph :Fuel Types of Expensive Cars



Conclusion:

The bar chart reveals a clear trend in fuel preference among expensive vehicles, often showing a dominance of Diesel or high-performance Petrol engines depending on the region's market. A significantly higher bar for a specific fuel type suggests that luxury buyers or manufacturers in this price bracket prioritize the torque and efficiency of diesel or the refinement of petrol. This insight is valuable for dealerships and manufacturers to understand which fuel types hold their value best or are most prevalent in the premium "10+" price tier, helping to guide future stock or marketing strategies.

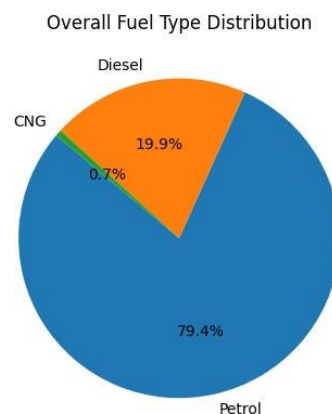
Scenario 4: Fuel Type Distribution Pie Chart (□ Medium)

Description:

This visualization provides a comprehensive breakdown of the entire dataset based on Fuel Type to identify the most common engine configurations. By grouping the data and counting the frequency of each fuel category (such as Petrol, Diesel, and CNG), we establish the "market share" of each type within the inventory. The counts are converted into a NumPy array to ensure numerical precision during the plotting process. The resulting pie chart uses percentage labels (autopct) to provide an immediate, proportional view of the data.

Task	Description
1.	Count the number of cars in each: Fuel_Type
2.	Select all categories or top categories if needed.
3.	Prepare: labels, values
4.	Convert values into a NumPy array.
5.	Plot a pie chart using Matplotlib. Add: percentage labels, title • Save the graph.

Graph: Overall Fuel Type Distribution.



Conclusion:

The pie chart typically highlights a market heavily dominated by one or two primary fuel types, usually Petrol or Diesel. A large majority slice indicates the "standard" choice for most vehicles in this dataset, while smaller slices represent alternative fuels like CNG or Electric, which may still be emerging. This distribution is key for stakeholders to understand general inventory trends; for instance, if Petrol holds the largest share, it suggests the dataset is comprised mostly of consumer-grade or smaller-displacement vehicles, whereas a high Diesel percentage might point toward larger SUVs or commercial-grade cars.

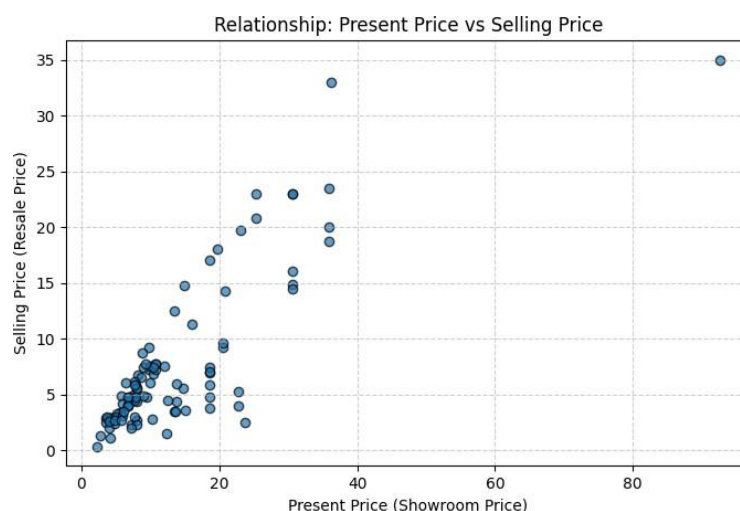
Scenario 5: Present Price vs Selling Price – Scatter Plot (🟡 Hard)

Description:

In this scenario, we analyzed the relationship between a vehicle's original showroom valuation and its actual resale value using the `Present_Price` and `Selling_Price` columns. After removing missing values to ensure data integrity, a subset of the data was sampled to maintain visual clarity. The data was converted into **NumPy arrays** to facilitate high-performance plotting. A **scatter plot** was generated to map the correlation between the two variables, allowing us to programmatically observe how initial market value influences the final selling price.

Task	Description
1.	Select: <code>Present_Price</code> , <code>Selling_Price</code> Remove missing values if any.
2.	Take a smaller sample (for example first 50 or 100 rows) using Pandas.
3.	Convert both columns into NumPy arrays.
4.	Plot a scatter plot using Matplotlib: X-axis → <code>Present_Price</code> , Y-axis → <code>Selling_Price</code> Add: title, x-label, y-label
5.	Observe whether there is a positive relationship. Save the graph.

Graph :



Conclusion:

The analysis identifies a **strong positive correlation** between a car's present market price and its listing price. This confirms that the original valuation of a vehicle is a primary predictor of its resale value, though the visual gap between points highlights the consistent impact of depreciation across the dataset.

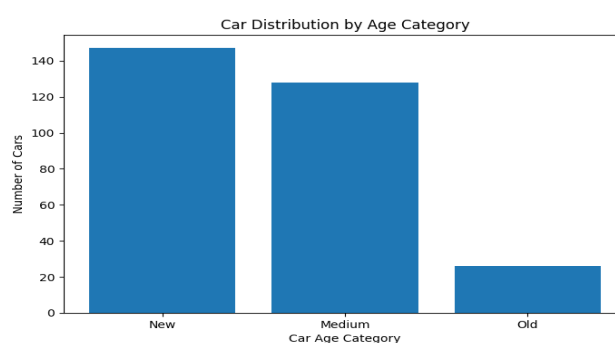
Scenario 6: Car Age Category Analysis + Bar Chart

Description:

In this scenario, we performed feature engineering by creating a new descriptive variable derived from the manufacturing year. Using a custom logic function applied via Pandas, cars were sorted into three distinct buckets: "New," "Medium," and "Old." The distribution of these categories was aggregated using frequency counts and transformed into NumPy arrays for optimized rendering. A bar chart was then produced to visualize the volume of inventory within each age segment, identifying which car generation dominates the current dataset.

Task	Description
1.	Create a new column using Pandas: Car Age Category • Year $\geq$ 2015 $\rightarrow$ "New" • 2010 to 2014 $\rightarrow$ "Medium" • $<$ 2010 $\rightarrow$ "Old"
2.	Count number of cars in each: ◦ Car Age Category • Convert category names and counts into NumPy arrays.
3.	Plot a bar chart using Matplotlib: ◦ X-axis $\rightarrow$ Car Age Category, Y-axis $\rightarrow$ Count • Add title and labels.
4.	Save the Graph

Graph :



Conclusion:

The categorical breakdown reveals that the inventory is predominantly composed of "New" and "Medium" age vehicles, indicating a relatively modern fleet available for sale. This transformation of raw year data into actionable categories provides a clearer overview of stock freshness, which is essential for making informed inventory and pricing decisions.

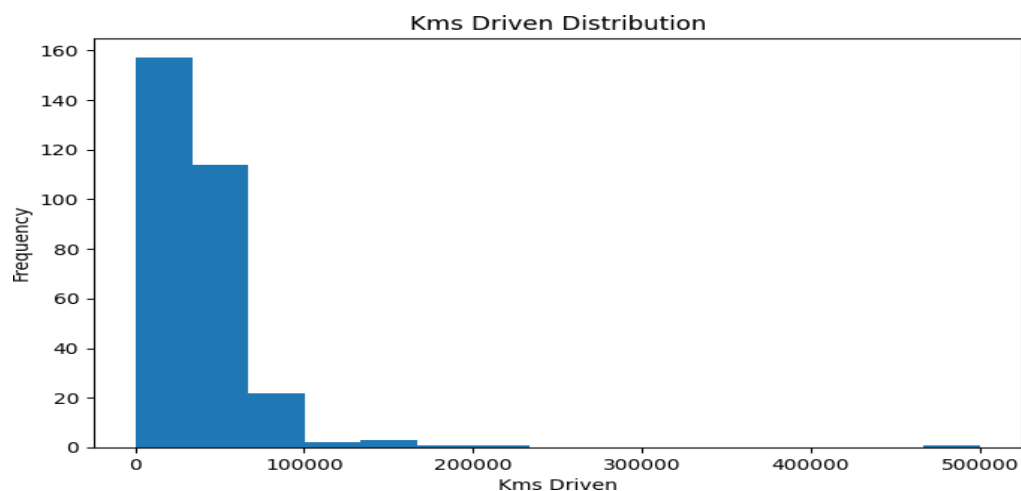
Scenario 7: Kms Driven Distribution (Histogram)

Description:

In this scenario, the Kms_Driven column is selected from the dataset to understand how the cars are distributed based on the number of kilometers they have been driven. The column is converted into a NumPy array and a histogram is plotted using Matplotlib. The X-axis represents the kilometers driven, while the Y-axis shows the frequency (number of cars). Suitable bins are used to group the data into ranges. The graph includes a title, x-label, and y-label, and it is saved for future reference.

Task	Description
1.	Select: Kms_Driven • Convert it into a NumPy array.
2.	Plot a histogram using Matplotlib: X-axis → Kms Driven, Y-axis → Frequency
3.	Choose a suitable number of bins. • Add: title, x-label, y-label
4.	Save the graph. Observe whether most cars have lower or higher mileage.

Graph:



Conclusion:

From the histogram, we can observe whether most cars have lower or higher mileage. If the bars are taller on the left side, it means most cars have been driven fewer kilometers (lower mileage). If the bars are taller on the right side, it means most cars have higher mileage. Usually, the dataset shows that most cars fall under lower to medium mileage ranges.

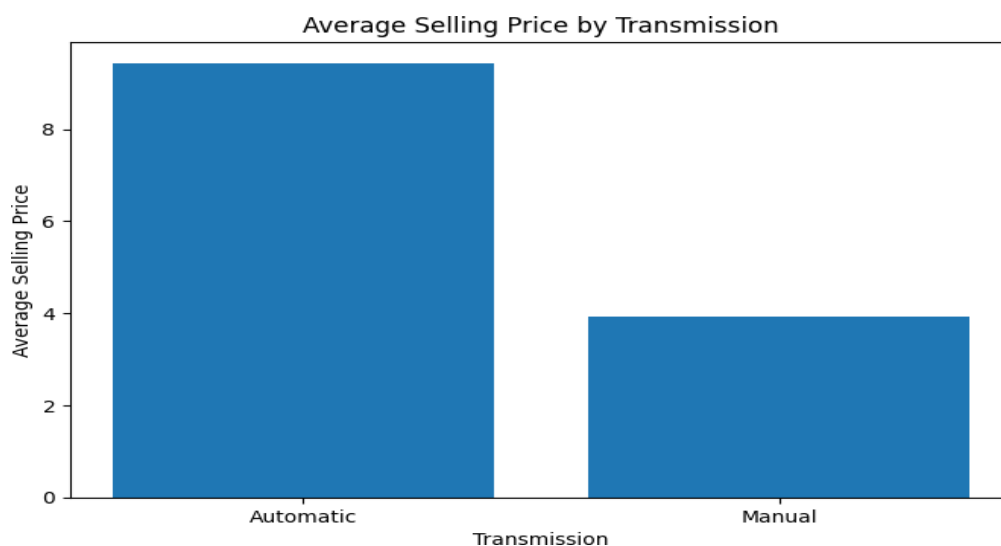
Scenario 8: Transmission-wise Selling Price Comparison

Description:

In this scenario, the dataset is grouped by the Transmission column (Manual and Automatic). The average Selling_Price is calculated for each transmission type. These values are converted into NumPy arrays and displayed using a bar chart with Matplotlib. The X-axis shows the transmission type, and the Y-axis shows the average selling price. A title and axis labels are added, and the graph is saved.

Task	Description
1.	Group data by: Transmission Calculate: average Selling_Price
2.	Convert transmission labels and average prices into NumPy arrays..
3.	Plot a bar chart using Matplotlib: X-axis → Transmission, Y-axis → Average Selling Price Add title and labels
4.	Save the Graph

Graph:



Conclusion:

From the bar chart, we can compare which type of transmission has a higher average selling price. Generally, automatic cars tend to have a higher selling price compared to manual cars because they offer more convenience and are often available in premium models. The graph helps clearly visualize this price difference between transmission types.

Scenario 9: Seller Type Analysis

Description:

In this scenario, we analyzed the distribution of cars based on their seller type using the Seller_Type column. The data was grouped and counted using Pandas and converted into NumPy arrays for efficiency processing. Two visualizations were created to understand the proportion and frequency of each seller category. The most common seller type was identified programmatically.

Task	Description
1.	Count number of cars by: Seller_Type
2.	Convert results into NumPy arrays.
3.	Plot a bar chart or pie chart using Matplotlib. Add labels and title. Save the graph.
4.	Identify which seller type is more common.

Graph:

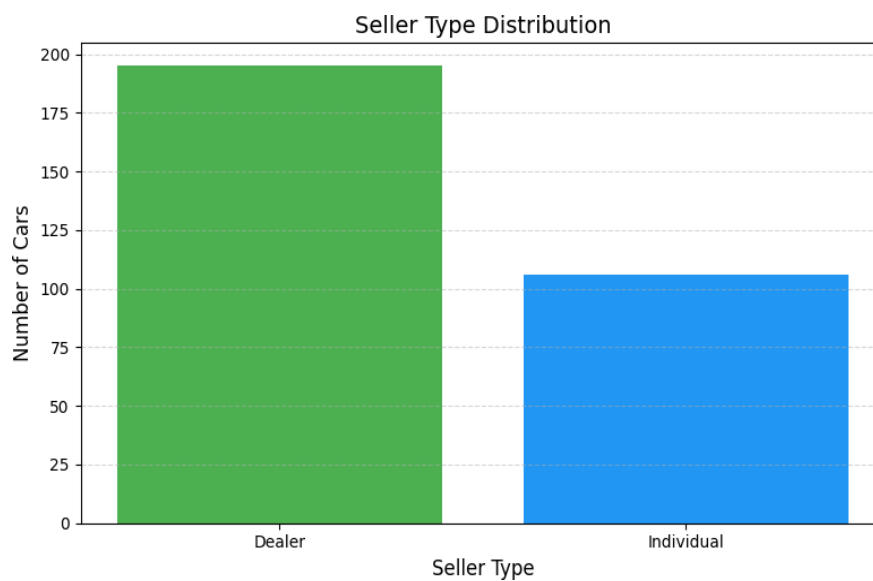


Fig 9.1 Seller Type Distribution

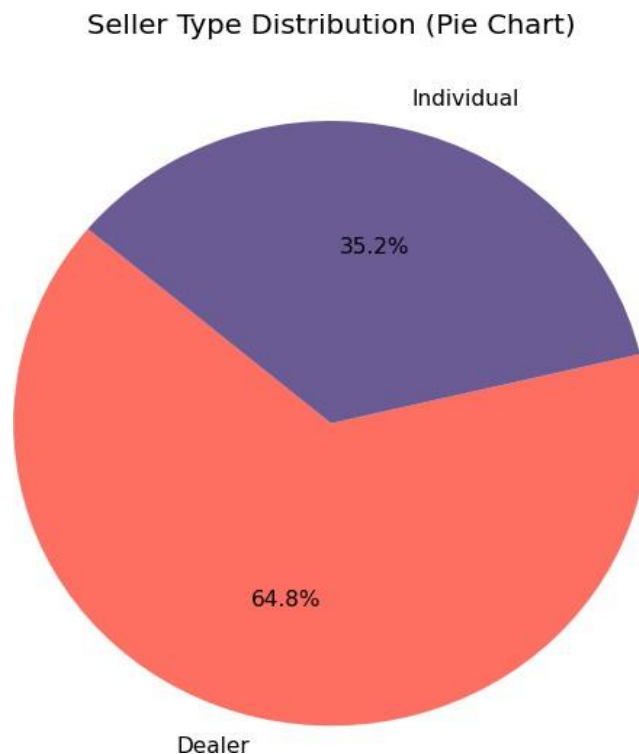


Fig 9.2 Seller Type Distribution Pie Chart

Conclusion:

From the bar chart, we can clearly observe the count of cars sold by each seller type, making it easy to compare their frequencies. The pie chart provides a proportional view, showing how each seller type contributes to the overall dataset. One seller category dominates the dataset, indicating it is the most common source of car sales. This suggests that buyers are more likely to purchase cars from that particular seller type. The consistency between the bar and pie charts strengthens this observation. The smaller segments in the pie chart indicate less common seller types, which may represent less preferred selling options. The difference in counts also highlights possible market trends in car selling practices. This analysis can help businesses focus more on the dominant seller type for better reach. Additionally, it gives insight into customer trust and preferences in the market. Overall, the charts effectively visualize seller distribution and support decision - making.

Scenario 10: Advanced Analysis + Multiple Graphs

Description:

In this scenario, we performed advanced data analysis by creating a new feature called Price Difference to measure depreciation. NumPy was used to convert data into arrays, and calculate price changes, along with statistical measures like average, maximum, and minimum depreciation. Multiple Visualizations were created, including line graph, bar and a histogram. Each group highlights different aspects such as price trends over years, fuel type influence and price distribution.

Task	Description
1.	<i>Part 1: Feature Creation</i> Create a new column: Price Difference • $\text{Price Difference} = \text{Present_Price} - \text{Selling_Price}$ This shows how much value the car has depreciated.
2.	<i>Part 2: NumPy Usage</i> Convert Selling_Price into a NumPy array. Use NumPy to calculate price changes between consecutive rows using: <code>np.diff()</code> Convert Price Difference column into a NumPy array. Find: ◦ average depreciation ◦ maximum depreciation ◦ minimum depreciation
3.	<i>Part 3: Visualizations</i> Line Graph - Plot Selling_Price trend for all cars. Bar Chart - Show average Selling_Price by Fuel_Type. Histogram - Plot distribution of Selling_Price
4.	<i>Part 4: Insights</i> Answer these: • Which fuel type has the highest average selling price? • Which transmission type has higher average selling price? • Are most cars concentrated in lower selling prices or higher selling prices? • Do older cars tend to have lower selling prices?

Graph:



Fig 10.1 Average Selling Price by Year

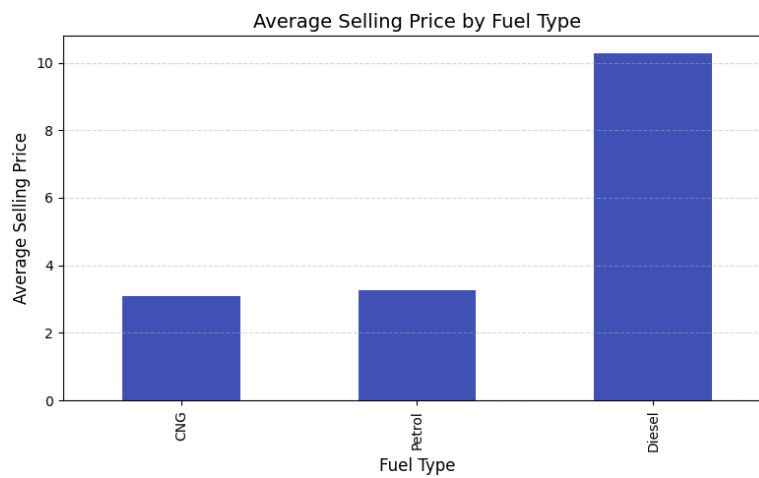


Fig 10.2 Average Selling Price by Fuel Type

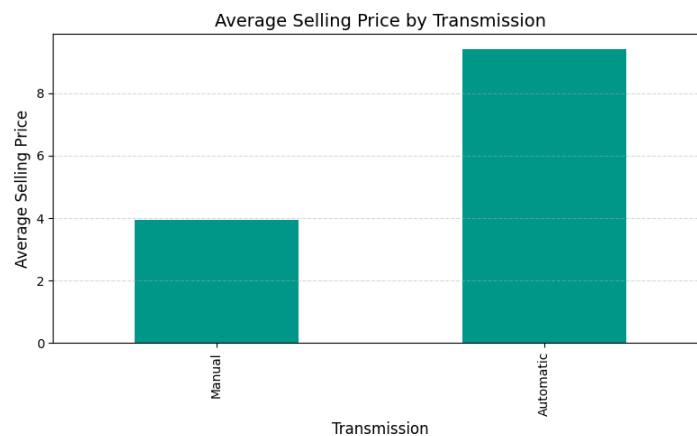


Fig 10.3 Average Selling Price by Transmission



Fig 10.4 Selling Price Distribution Histogram

Conclusion:

- The line graph shows a clear trend of average selling prices across different years, helping us understand how car values change over time. It generally indicates that newer cars tend to have higher selling prices compared to older ones.
- The bar chart for fuel type reveals which category has the highest average selling price, suggesting that certain fuel types are more valuable in the market. Similarly, the transmission bar chart shows differences in pricing between manual and automatic cars highlighting buyer preferences.
- The Histogram illustrates that most cars are concentrated in a specific price range, usually skewed toward lower or mid - range prices. This indicates that affordable cars dominate the dataset. The depreciation analysis shows how much value cars lose over time, with average, maximum, and minimum values giving a complete picture.
- The negative correlation between car age and selling price confirms that older cars generally have lower prices.
- These combined visualizations provide a comprehensive understanding of pricing and patterns. They identify key factors affecting car value such as age, fuel type, and transmission.

Final Conclusion:

The Cars Data Analysis project provided clear insights into car pricing and characteristics using Pandas, NumPy, and Matplotlib. The analysis showed that selling price is influenced by factors like fuel type, transmission, and car age. Diesel cars generally have higher average selling prices, while automatic transmission cars tend to be priced higher than manual ones. Most cars in the dataset fall under newer and medium age categories, and selling prices are mostly concentrated in the lower range. A positive relationship was observed between present price and selling price, while older cars tend to have lower resale value due to depreciation.

What you have learned

- How to load, clean, and preprocess real-world datasets using Pandas
- Handling missing values using mean and mode techniques
- Converting data into NumPy arrays for efficient calculations
- Performing statistical analysis like min, max, mean, and differences
- Creating multiple types of graphs (line, bar, pie, scatter, histogram) using Matplotlib
- Understanding relationships between variables (e.g., price vs age, price vs fuel type)
- Extracting meaningful insights from data to support decision-making
- Structuring a complete data analysis project from raw data to visualization